

SMM694 Applied Natural Language Processing

Session 3

Vector Semantics & Embeddings

Valerio Modugno | June 2026

Session Roadmap

1. Vector Semantics & Embeddings

2. word2vec

- word2vec basics
- The algorithm

3. Interpreting Word Vectors

- Scope of application
- Application examples

4. Classes & Examples of Embeddings

word2vec and beyond

5. Multimodal Embeddings & CLIP

Cross-modal architectures, contrastive learning, and zero-shot visual classification.

6. Conclusion

Vector Semantics and Embeddings

Goals: Vector Semantics & Embeddings

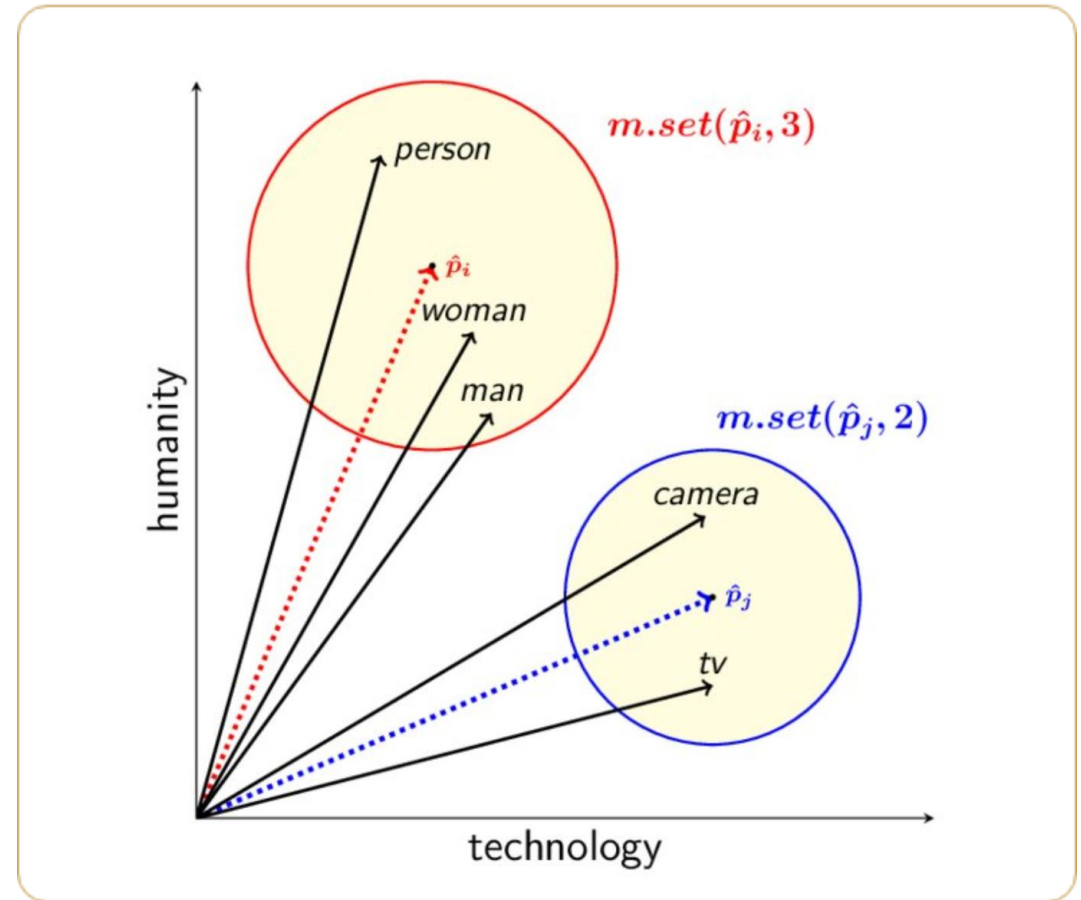
- ✓ Consolidate knowledge on the background and objectives of word vectors.
- ✓ Appreciate the importance of producing **dense real-value word vectors**.
- ✓ Familiarise yourself with the most popular word embedding algorithms.
- ✓ Explore how foundational semantic spaces can be expanded to bridge different modalities (text and images) using models like **CLIP**.

The Distributional Hypothesis

"Words appearing in similar contexts have similar meanings."

Example: The words "dog" and "bark" are related because they share similar contexts.

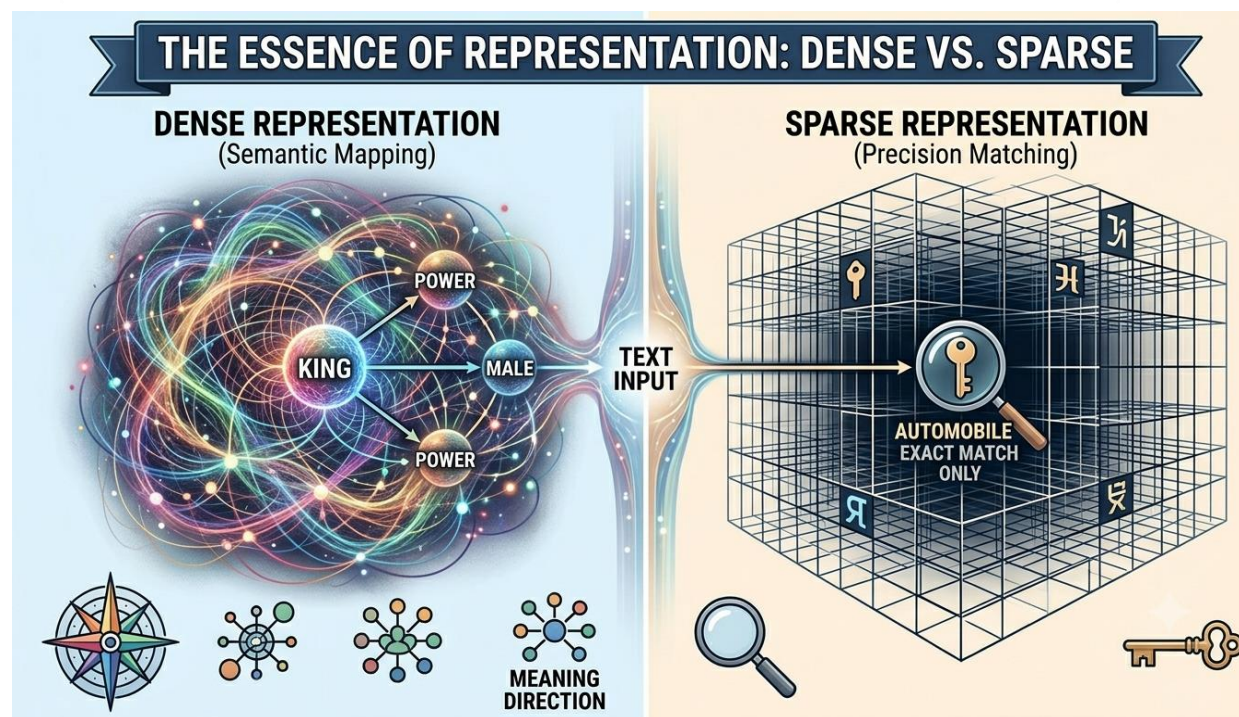
- ✓ **Machine Learning (ML)** generates dense, real-value word vectors.
- ✓ Embeddings represent a word in a sense vector.
- ✓ Effectively captures synonymy and semantic similarity.



Why Dense Real-Value Vectors?

- ✓ **Convenience:** Dense vectors reduce the number of features in Machine Learning tasks compared to sparse arrays.
- ✓ **Effectiveness:** These vectors inherently capture semantic similarities.

Example: "car" and "automobile" should have similar vectors because they are synonyms.



Examples of Word Embeddings (1)

word2vec

A pioneer field that trains by predicting surrounding words in a context.

- Useful for morphologically rich languages by aggregating sub-words.
- **Example:** German word "Straßenkehrer" (street sweeper) treated as a single entity.
- Easily implemented using libraries like Gensim.

fastText

Builds on word2vec by including subword information to generate vectors.

- Effective for handling out-of-vocabulary words.
- **Example:** Understands variations like "cat", "cats", and "catlike".

Examples of Word Embeddings (2)

GloVe

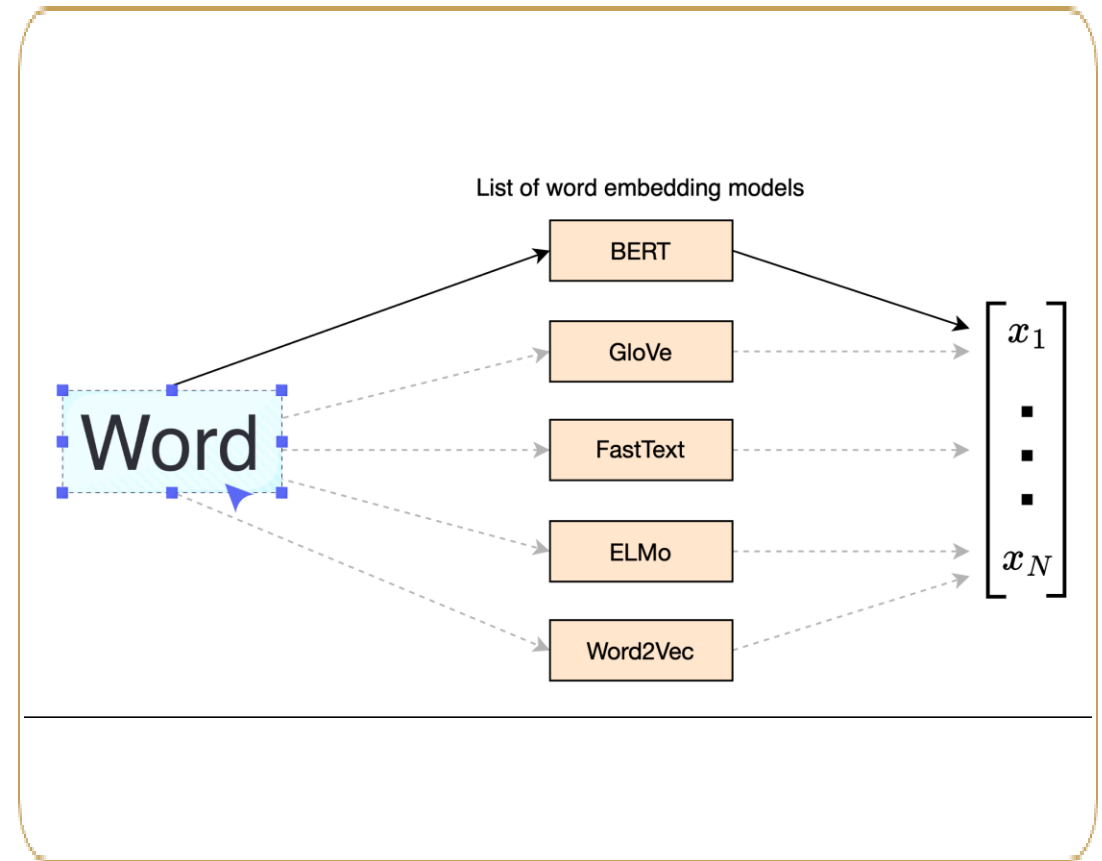
Focuses on the global context by analyzing word co-occurrence in a corpus using a matrix.

Example: "king" and "queen" have similar vectors due to related contexts.

Contextual Embeddings

Vectors change based on context, unlike static embeddings (e.g., ELMo, BERT).

Example: "bank" in "river bank" vs. "financial bank".



Word2vec



Goals: word2vec

- ✓ Appreciate the anatomy of the word2vec algorithm.
- ✓ Understand the various steps required to train ad hoc embeddings.
- ✓ Assign a role to new forms of embeddings relative to this foundational model.

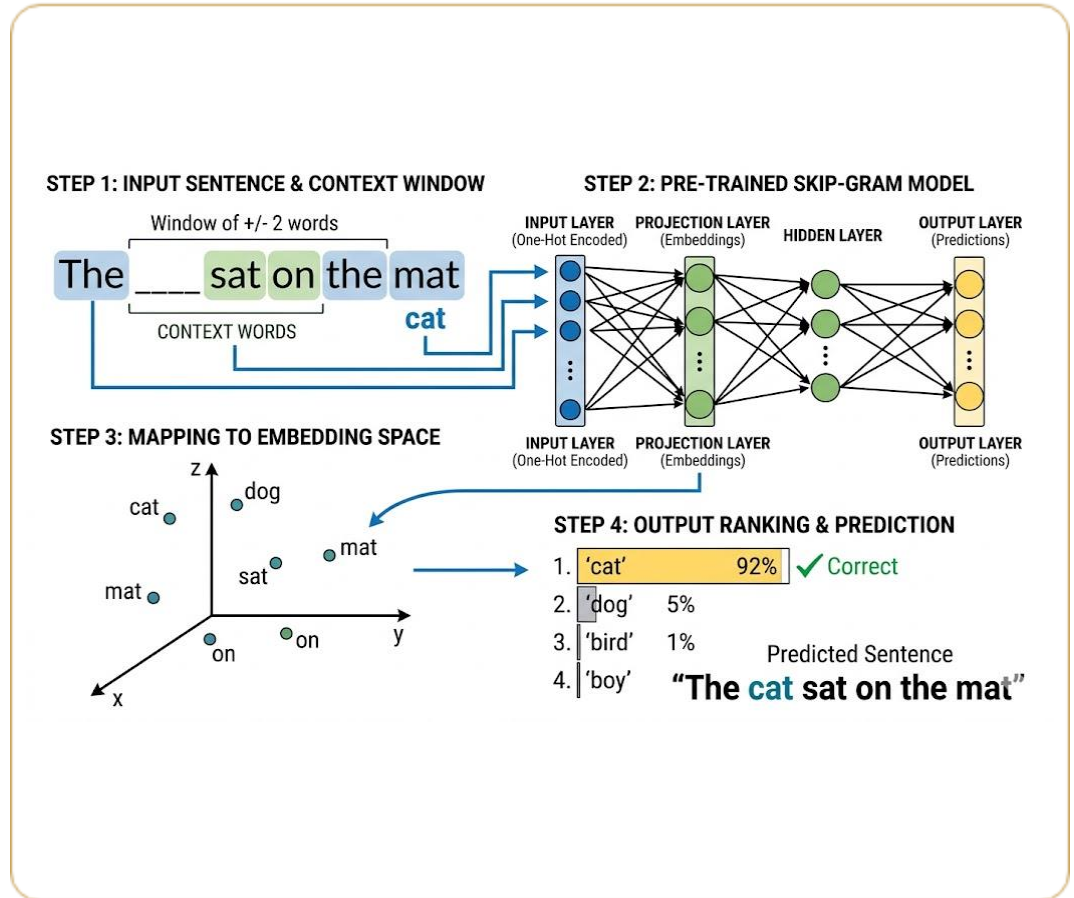
Introduction to word2vec

Introduced in 2013, word2vec marked a significant shift in NLP.

- ✓ No hand-labeled supervision required.
- ✓ Fast training capabilities.
- ✓ Open-source availability.

Philosophy: Train a classifier on predicting the likelihood of context words.

Example: Predicting the word "cat" in the context of "The __ sat on the mat."



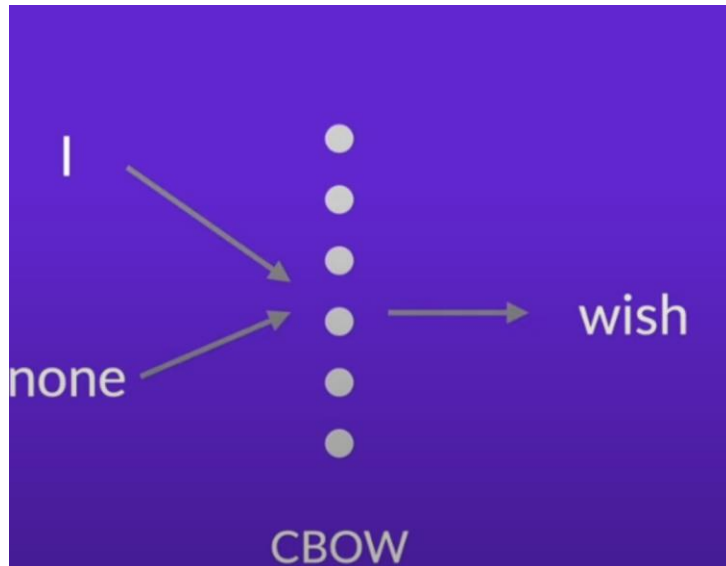
Word2vec Architectures

"... I **wish** none of this has happened. So do all who live to see such times..."

CBOW (Continuous Bag of Words)

Maps multiple context words to predict a single target word.

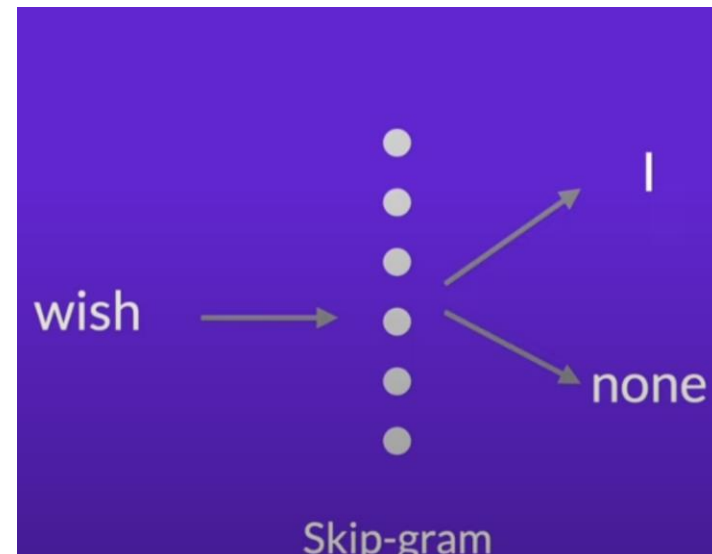
e.g., Predicting "wish" from surrounding words.



Skip-gram

Maps a single target word to predict multiple surrounding context words.

e.g., Using "wish" to predict "none" and others.



Skip-Gram Algorithm (Part 1)



Positive Examples

Treats the target word and neighboring context words as positive examples.



Negative Sampling

Randomly samples other words in the lexicon to serve as negative examples.

Skip-Gram Algorithm (Part 2)

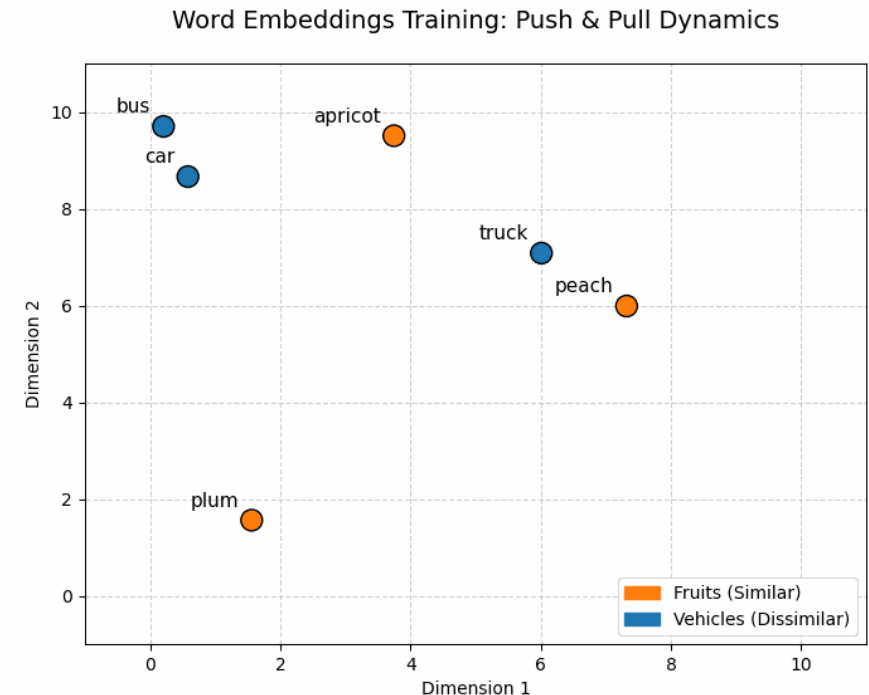
Uses logistic regression to train a classifier to distinguish between positive and negative cases.

The weights derived from this process are used as the embeddings.

Target: 'apricot' (window size 2)

Positive Examples: ('apricot', 'fruit'), ('apricot', 'sweet')

Negative Examples: ('apricot', 'car'), ('apricot', 'building')



Skip-Gram Training Objectives

- ✓ Training data consists of a target word (like 'apricot') and its context lexemes based on a window of length '2'.
- ✓ Words are initially represented as **randomly initialized vectors** of a certain length (e.g., 300).

Maximize Similarity

Aim to maximize the similarity of target and context word pairs (positive data).

Minimize Similarity

Aim to minimize the similarity of pairs drawn from negative data.

Skip-Gram Probability Estimation

The goal is to estimate the probability that a context word c is a real context word for a target word t .

Softmax Function

$$P(c | t) = \frac{\exp(t \cdot c)}{\sum_{c' \in V} \exp(t \cdot c')}$$

Where t and c are target and context word vectors, and V represents the vocabulary.

Word2vec Training

Converting Text to Numbers



INGEST CORPUS

Read massive datasets (e.g., Wikipedia).
The scale provides the necessary
statistical distribution for meaning.



BUILDING VOCABULARY

Read massive datasets (e.g., Wikipedia).
The scale provides the necessary
statistical distribution for meaning.



ONE-HOT ENCODING

Read massive datasets (e.g., Wikipedia).
The scale provides the necessary
statistical distribution for meaning.

Mapping Semantic Context

-  Sliding Window: A fixed window (e.g., 2 words) slides through the text sequentially.
-  Target & Context: The center word is the 'Target', and neighbors are the 'Context'.
-  Pair Generation: For "quick brown fox", the algorithm extracts pairs like (brown, quick) and (brown, fox).

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
not	machine
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

The Hidden Architecture



INPUT LAYER

Accepts the **10,000-dimensional** one-hot vector representing the Input Word.



HIDDEN LAYER

The Embedding Matrix (**10k x 300**).
Initialized with random weights and no activation.



OUTPUT LAYER

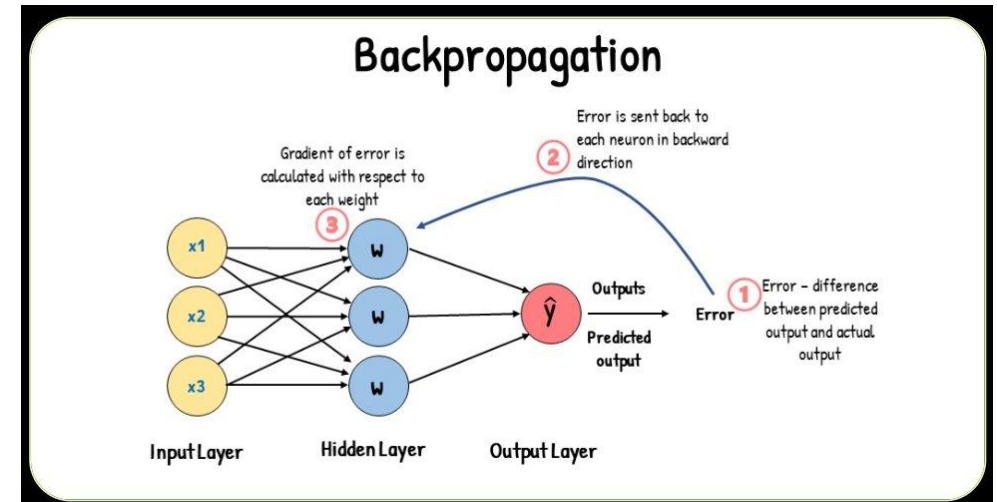
The Context Matrix. Maps the 300-dim vector back to the 10k vocabulary space.

Predicting the Context

-  **The Lookup:** One-hot input multiplies the Hidden Matrix to extract a specific 300-dim row.
-  **The Projection:** The word vector is projected onto the Output Matrix to produce raw context scores.
-  **Softmax Normalization:** Scores are squashed into probabilities summed to 1.0.

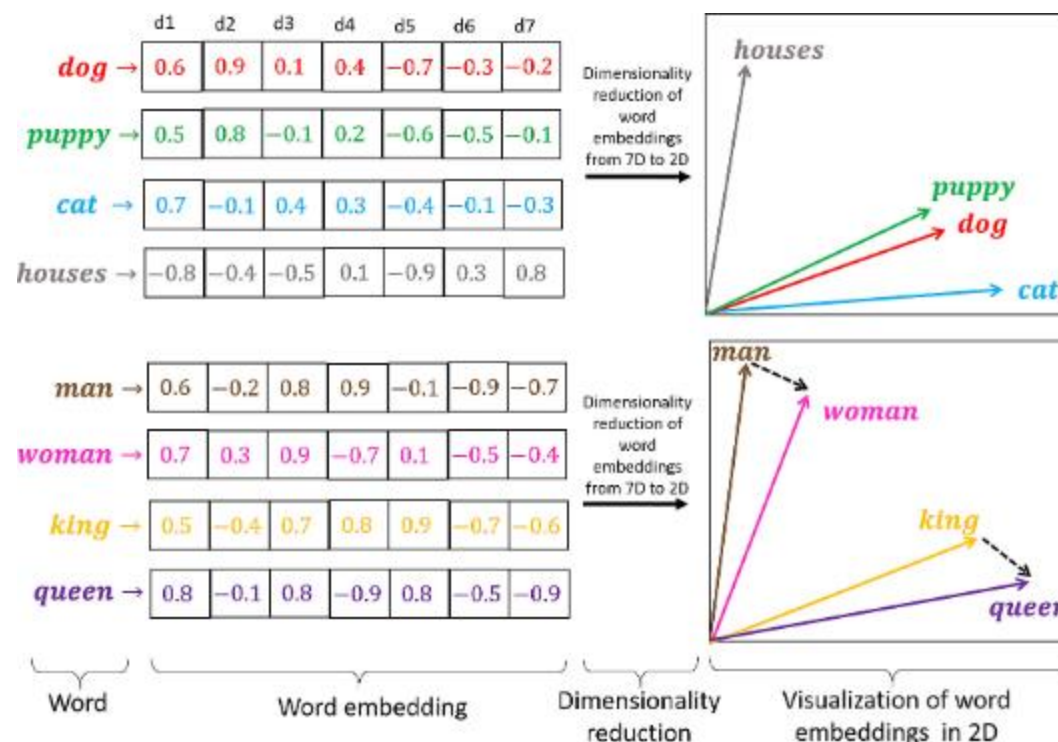
Learning From Error

- ❗ **Calculate Loss:** Cross-Entropy loss penalizes the model for low probabilities assigned to the correct context word.
- ⏮ **Backpropagation:** Compute the gradient of the loss to see how each weight contributed to the error.
- 📈 **Weight Update:** Slightly tweak the Hidden and Output matrices to favor the correct word in the next iteration.



Wrap-Up: word2vec

- ✓ Draws on the Distributional Hypothesis and Machine Learning.
- ✓ Predicts the likelihood of context words conditionally on the target word.
- ✓ Regresses conditional probabilities on real values included in the word vectors.
- ✓ During training, it adjusts these real values so that semantically similar words end up with similar vectors.



Interpreting Word Vectors

From mathematical creation to extracting analytical meaning and utility.

Goals: Interpreting Word Vectors

Core Objective

Appreciating the extensive **scope of application** of word embeddings.

***Perspective:** Word vectors are not merely abstract mathematical constructs; they serve as powerful instruments with real-world analytical uses, reflecting human language and culture.*

Traversing Vectors to Find Related Words

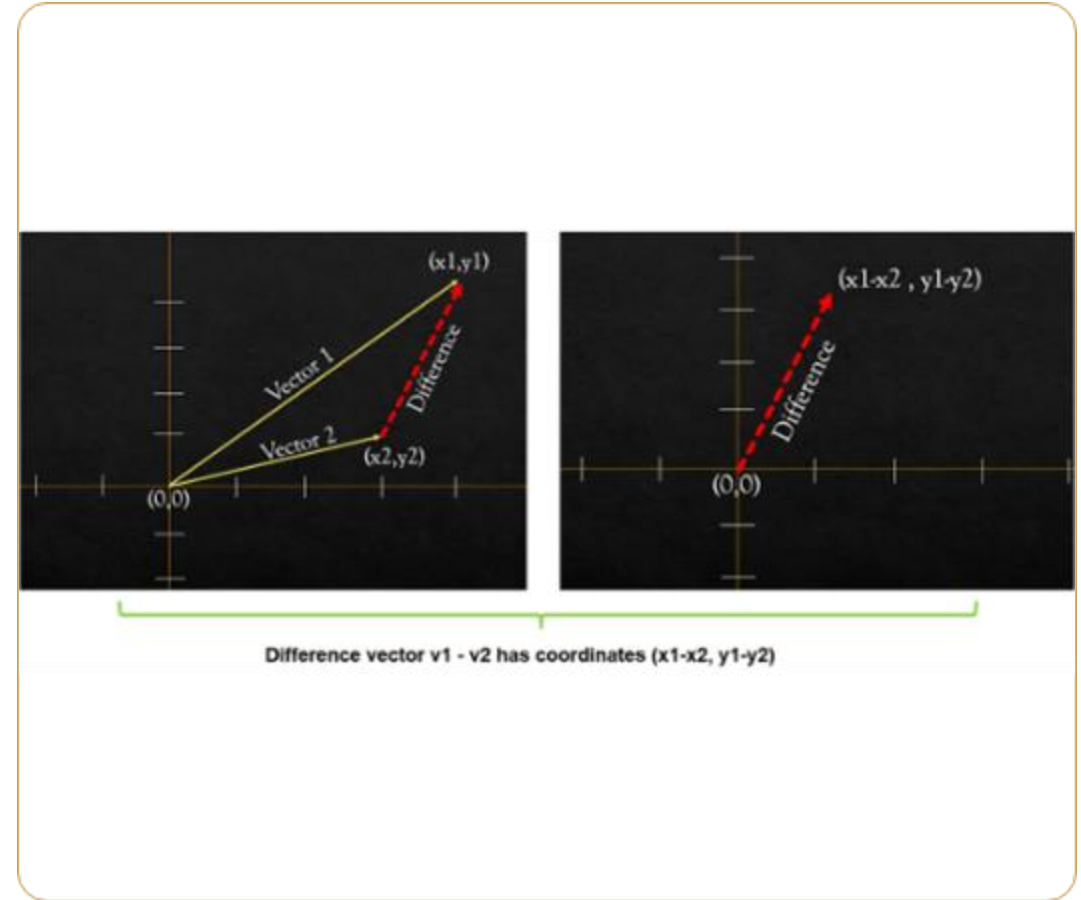
Linguistic or semantic similarity can be quantitatively measured using specific distance metrics within the vector space.

- ✓ **Euclidean Distance:** Measures the straight-line distance between two points.
- ✓ **Cosine Similarity:** Measures the angle between two vectors, focusing on orientation rather than magnitude.

Example Query: "frog"

Nearest Neighbors Returned: "toad", "lizard"

Because similar concepts cluster mathematically, querying a word reveals its closest semantic coordinate points.



Broad Scope of Application



Business & Economics

Analyzing market trends, consumer sentiment, and corporate communications through semantic shifts.



Cultural Analysis

Leveraging vast datasets (like Google Books) to map the evolution of language and culture over centuries.



Studying Historical Biases

Analyzing word usage in historical texts to detect and quantify changes in societal biases and stereotypes over time.



Bridging Modalities

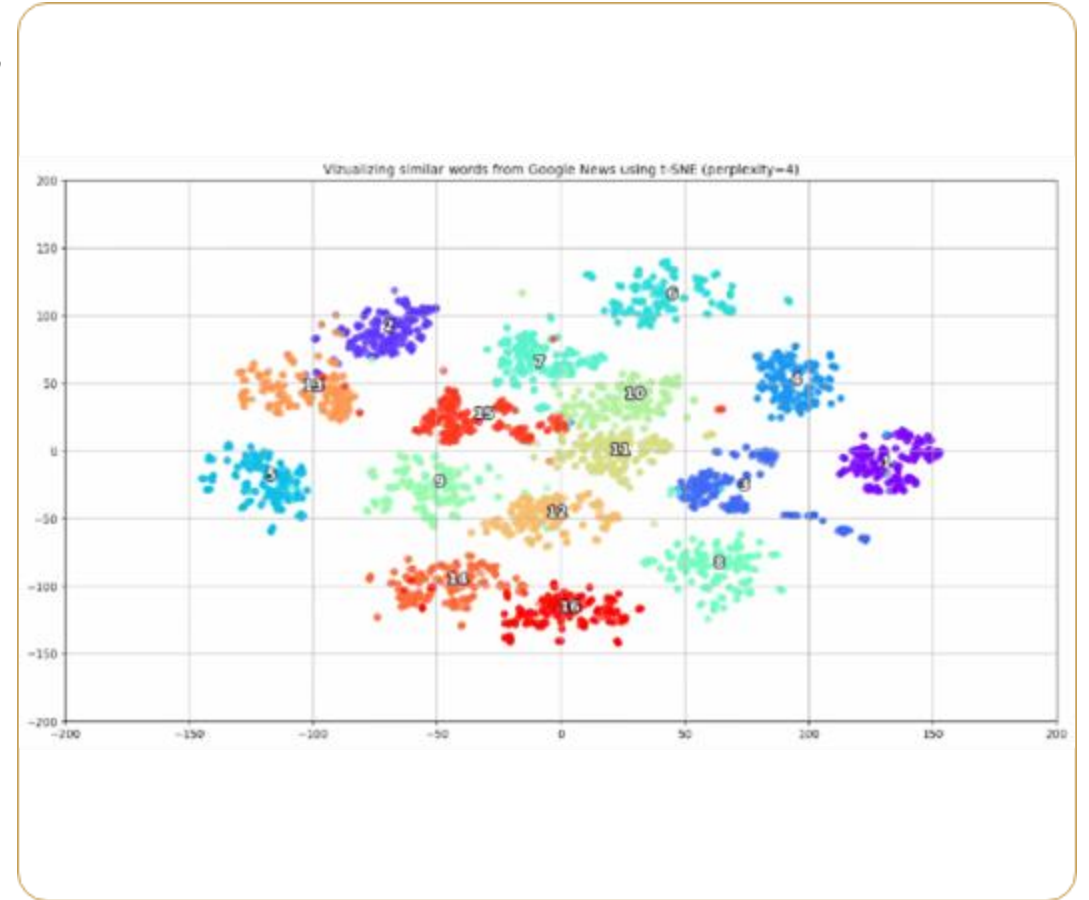
Connecting language and physical reality through multimodal shared spaces (e.g., controlling robots via natural language).

Visual Inspection of Embeddings

Vectors often exist in hundreds of dimensions (e.g., 300D), making them impossible to visualize natively.

- ✓ We must use **dimensionality reduction techniques**.
- ✓ **Methods:** PCA (Principal Component Analysis) or t-SNE.
- ✓ These techniques "squash" high-dimensional spaces down to 2D or 3D maps.

***Outcome:** By plotting vectors in 2D space, humans can literally see semantic clusters of related words and contextual changes.*



Embeddings as a Historical Tool

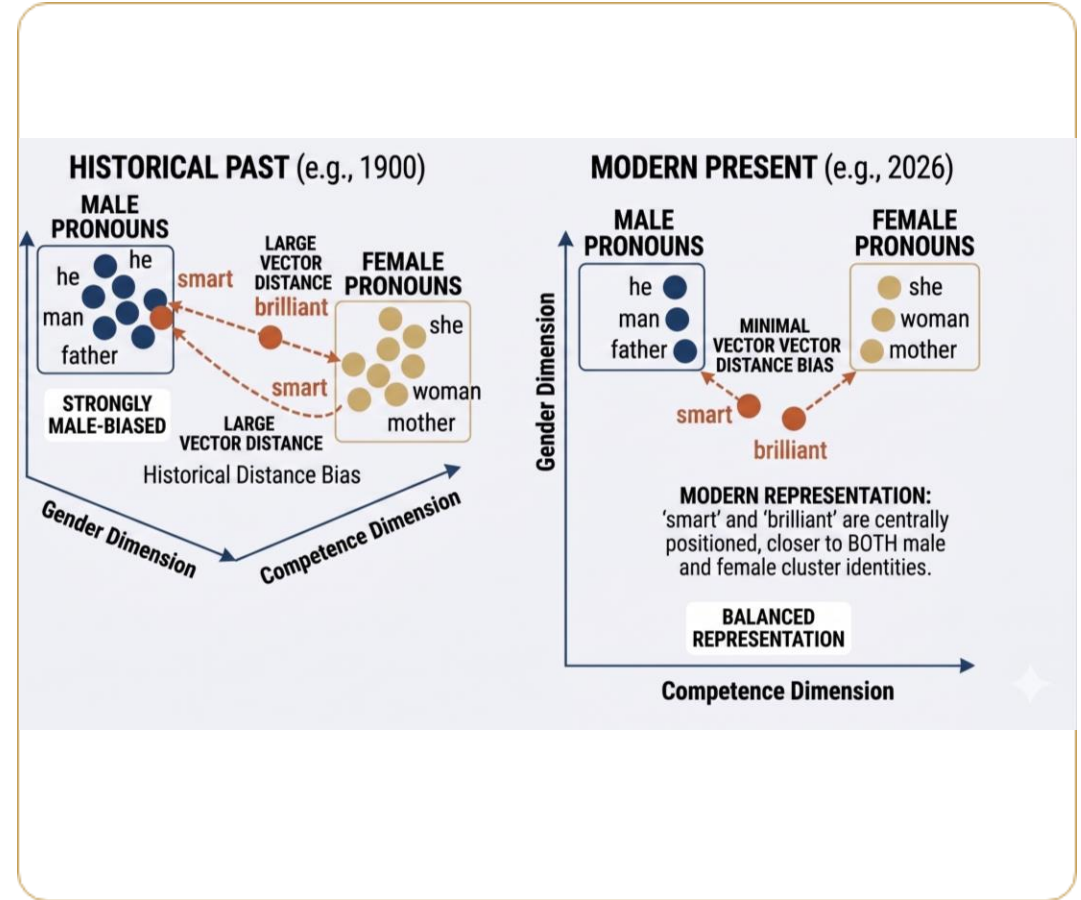
Embeddings map text patterns exactly as they occur, meaning they inherently encode human prejudices and historical context.

Case Study: Gender Bias

Examining the bias in competence adjectives like "smart", "wise", and "brilliant".

Observation: The vector distance historically showed a strong bias associating these terms with men, though this bias has been slowly decreasing over time.

Analysts can quantify historical sexism by measuring the mathematical distance between gendered pronouns and specific adjectives across different eras.



Wrap-Up: Interpreting Word Vectors

Discovering Latent Associations

Navigating word vectors allows analysts to mathematically discover latent semantic associations hidden within massive text corpora.

Reflecting Society

Word vectors reveal cultural stereotypes that affect the behavior of individuals, groups, and larger economic and social formations.

Word embeddings are not just linguistic tools, but sociological instruments that uncover hidden cultural truths embedded in text.

Classes & Examples of Embeddings



Exploring the broader ecosystem of embeddings beyond just single words.

Goals: Classes & Examples

- ✓ Exploring new forms of embeddings across different data structures.
- ✓ Using **doc2vec** to clarify how the application scope of vectorization can be expanded.

Objective: *To demonstrate how the core vectorization philosophy (dense representations based on context) has successfully expanded to capture concepts much larger and more complex than single atomic words.*

Embeddings Expanding word2vec

The underlying logic of word2vec has been adapted to map various types of non-word entities into continuous vector spaces.

doc2vec

Embeddings for entire documents.

graph2vec

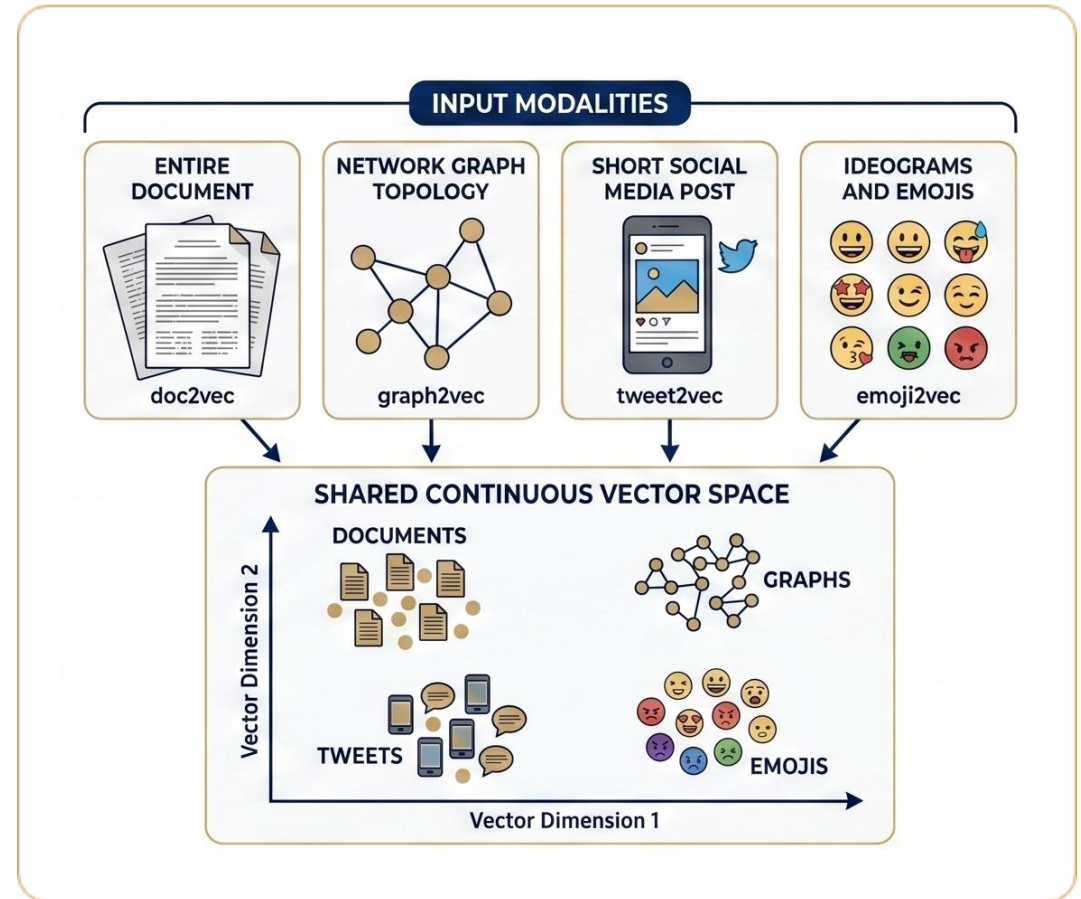
Embeddings for graph network topologies.

tweet2vec

Embeddings for short social media posts.

emoji2vec

Embeddings for ideograms and emojis.

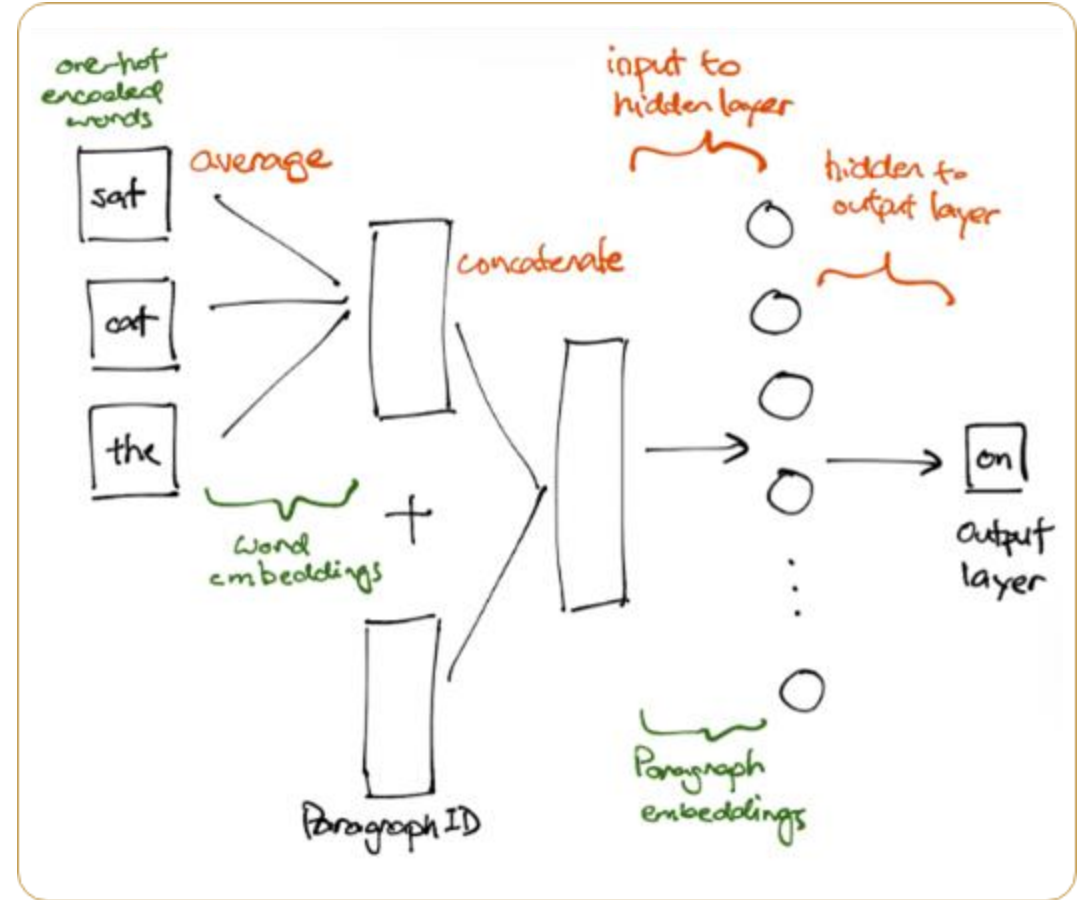


Embedding Documents: doc2vec

- ✓ doc2vec represents whole documents (paragraphs, articles) as single, dense vectors.
- ✓ It serves as a sophisticated alternative to the traditional Bag of Words (BoW) approach.

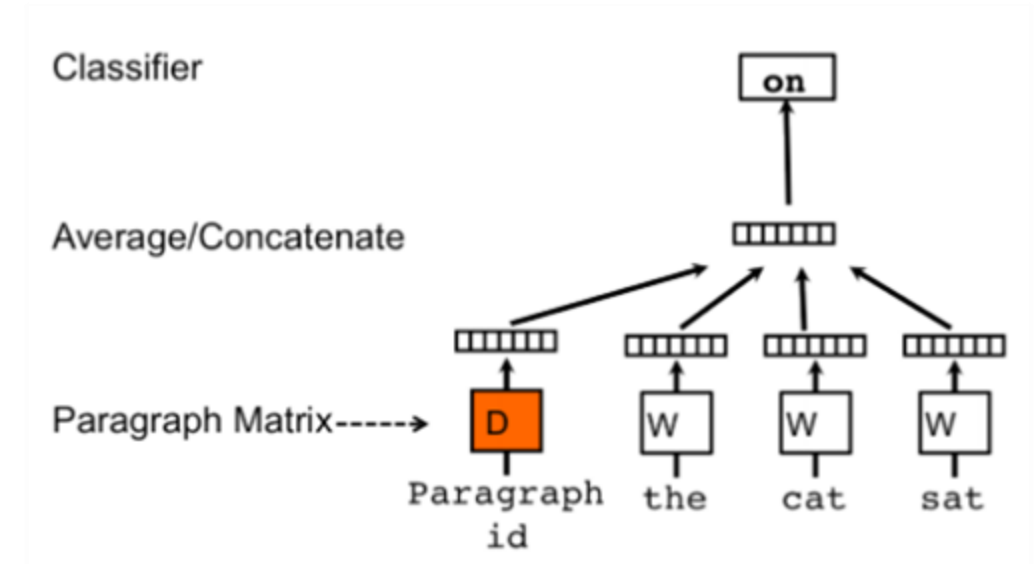
Example: Representing an entire news article as a single vector.

Instead of averaging thousands of word vectors, doc2vec creates a unified mathematical representation that encapsulates the overall theme and semantic thrust of the text.



The Machinery of doc2vec (PV-DM)

- ✓ The architecture modifies standard prediction models by introducing a unique **Paragraph ID** alongside standard word inputs.
- ✓ During training, one-hot encoded context words are mapped to standard word embeddings.
- ✓ Simultaneously, the document's ID is mapped to its own distinct paragraph embedding.
- ✓ These vectors are aggregated (averaged or concatenated), passed through a hidden layer, and used to predict the target word.



Distributed Memory: The Paragraph ID acts as a persistent memory component. While individual word vectors capture local syntax, the paragraph vector acts as a global "topic matrix," guiding predictions based on the document's broader context.

word2vec vs. doc2vec: Core Differences

word2vec Architecture

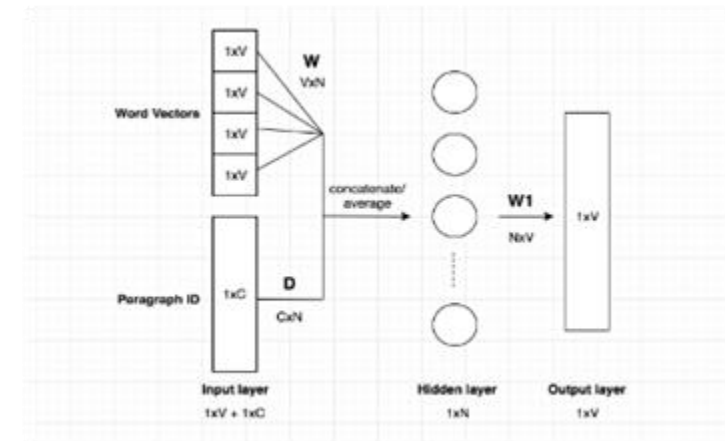
Learns vector representations purely by predicting a word based on a sliding window of a few surrounding words (Local Context). It assumes semantic meaning is entirely contained within immediate linguistic neighbors.

doc2vec Architecture

Operates on the same sliding window but forcibly injects a global document state. The model learns vocabulary weights globally, but document weights are uniquely optimized for each specific text block.

The Architectural Impact:

Simple word2vec treats the word "apple" the exact same way whether it appears in a technology blog or a fruit recipe. By training a dedicated, learnable vector for the document ID, doc2vec ensures the overarching narrative mathematically biases the prediction, creating a dense vector representing the whole document, not just the sum of its parts.



Multimodal Embeddings & CLIP



*Beyond text: Deep Dive into Contrastive Language Image
Pretraining*

The Limitation of Core Visual Models

Closed-World Assumptions

Traditional Computer Vision (CV) models are built on fixed-label classification, such as the ImageNet-1K benchmark. This architecture forces the model to predict from a pre-defined list of discrete integers.

Consequently, the model is unable to reason about concepts, styles, or specific nuances that fall outside its static training vocabulary.

The Operational Bottleneck

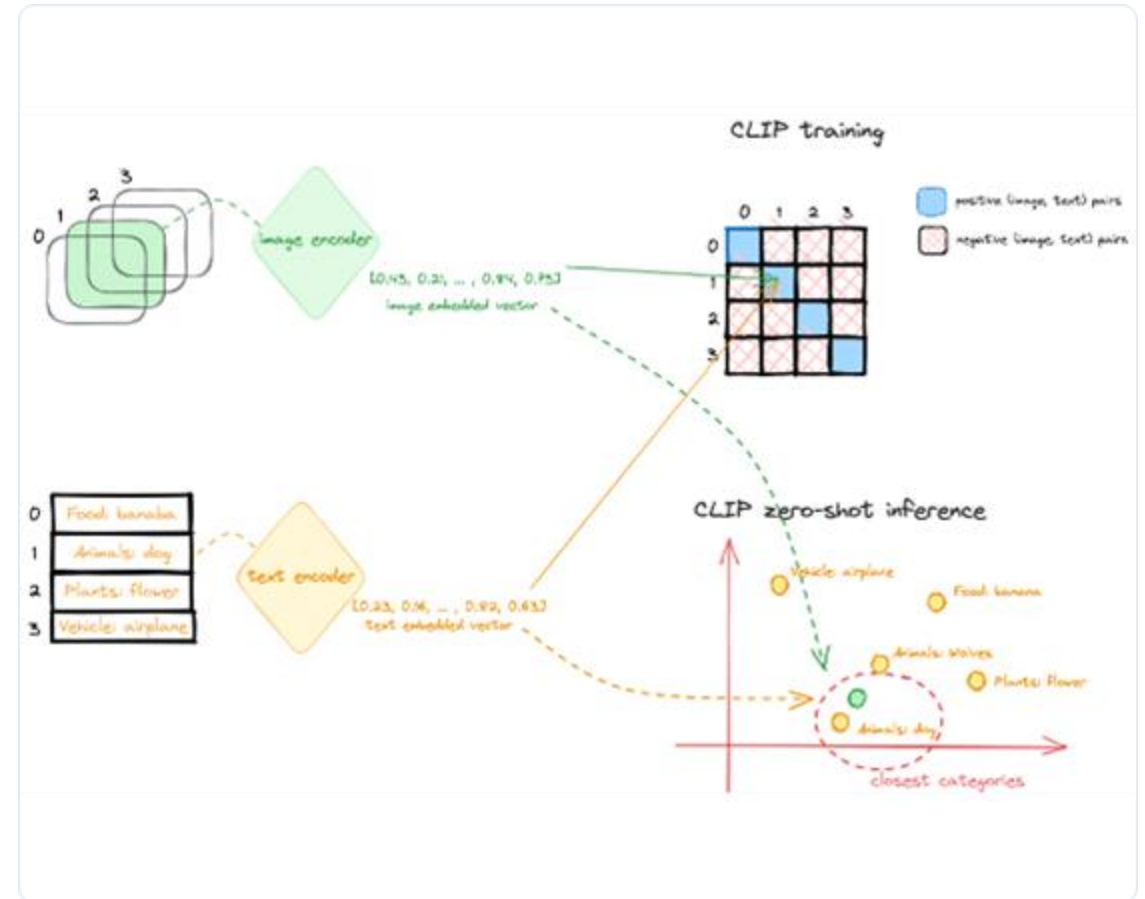
These models are "structurally deaf" to the rich semantics of natural language. A dog is labeled '0' and a cat '1'; the model understands they are different, but lacks the textual context that links "barking" to "dog" or "feline" to "cat."

We lose the semantic depth found in vector spaces when we reduce vision to a finite set of integer labels.

The Core Thesis of CLIP

Natural Language Supervision

Introduced by OpenAI in 2021, CLIP (Contrastive Language-Image Pretraining) flips the paradigm. Instead of integers, why not use natural language captions as our visual labels? The thesis is simple: Teach a model to match a **raw text caption** to its **visual counterpart**. This maps images and language into a single unified continuum where a "golden retriever playing in snow" shares identical mathematical coordinates with an image of that scene.



High-Level Architecture Overview

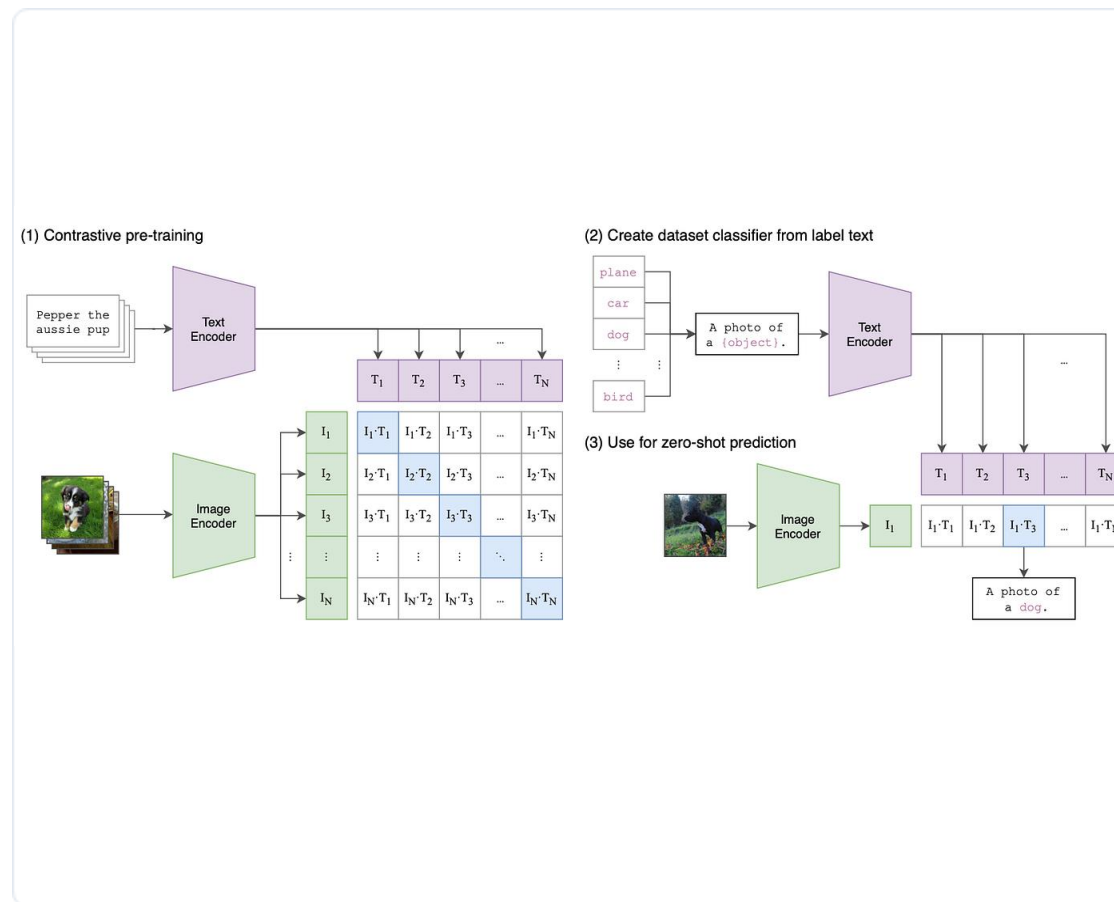
The Dual-Encoder Framework

CLIP operates via two parallel streams. The **Image Encoder** $g(I)$ processes visual pixels, while the **Text Encoder** $f(T)$

processes natural language strings.

The models don't interact during the encoding phase. Instead, they independently output raw feature representations.

These representations are then passed through linear projection layers to realign their dimensions into a **Shared Multi-modal Embedding Space** for comparison via dot product.



The Text Encoder Machinery

The Architecture

CLIP's text encoder uses a GPT-style Transformer (12-layer, 512-wide, 8 heads). Unlike simple models, it processes text as a sequence of tokens with complex spatial dependencies. This allows the model to handle long-range context and nuanced syntax across a maximum sequence length of 76 tokens.

Why not word2vec?

Static word vectors average everything out, destroying word order. In word2vec, "A dog chasing a car" and "A car chasing a dog" are mathematically identical because they share the same tokens.

The Transformer Advantage: Self-attention preserves syntax and positional relationships, mapping complex descriptions to specific visual scenes.

Image Encoder A: ResNet Modernization

Updating Legacy Architectures

One option for CLIP's visual backbone is the ResNet (50 or 101). However, OpenAI modernized it for multimodal tasks. They replaced the standard "Global Average Pooling" layer with **Attention Pooling**.

By placing a multi-head QKV (Query-Key-Value) attention layer at the end of the ResNet backbone, the model can dynamically weight specific visual regions before emitting its final vector representation, making it much more sensitive to textual cues.

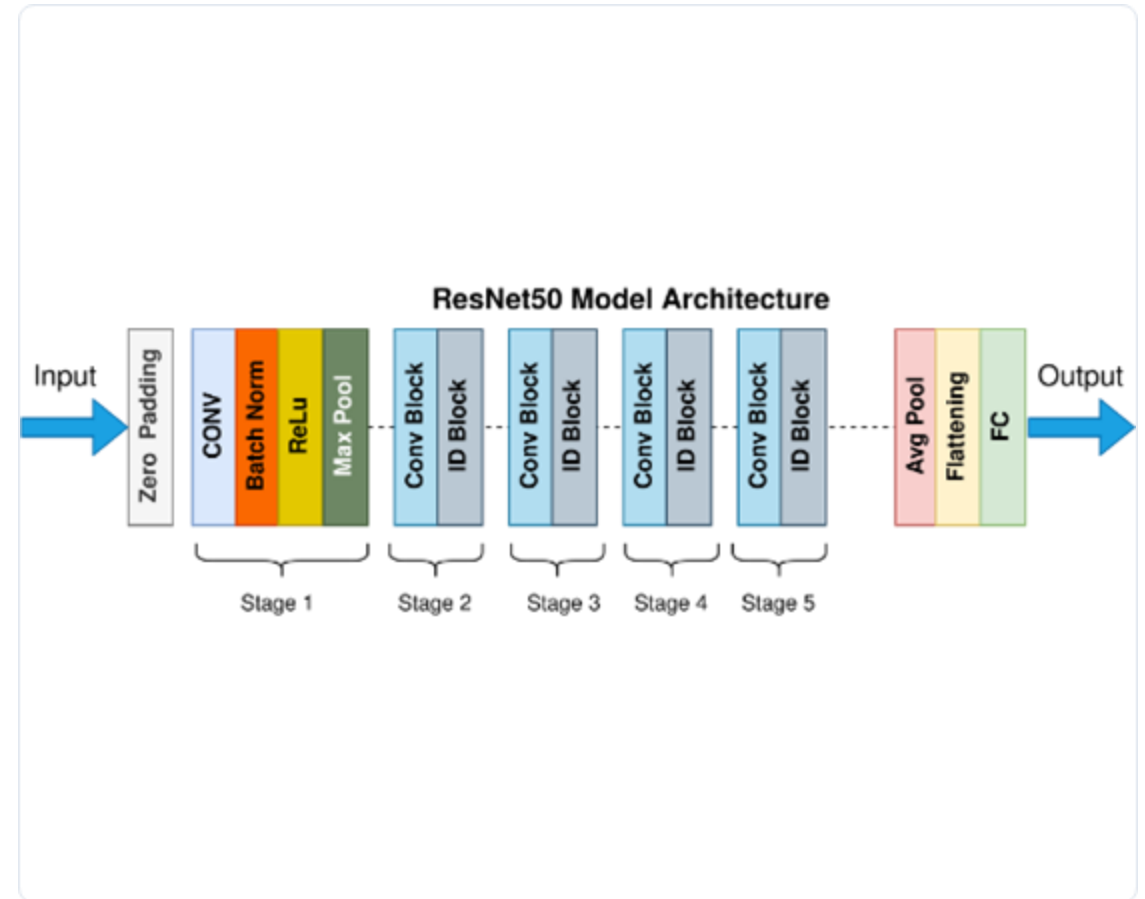
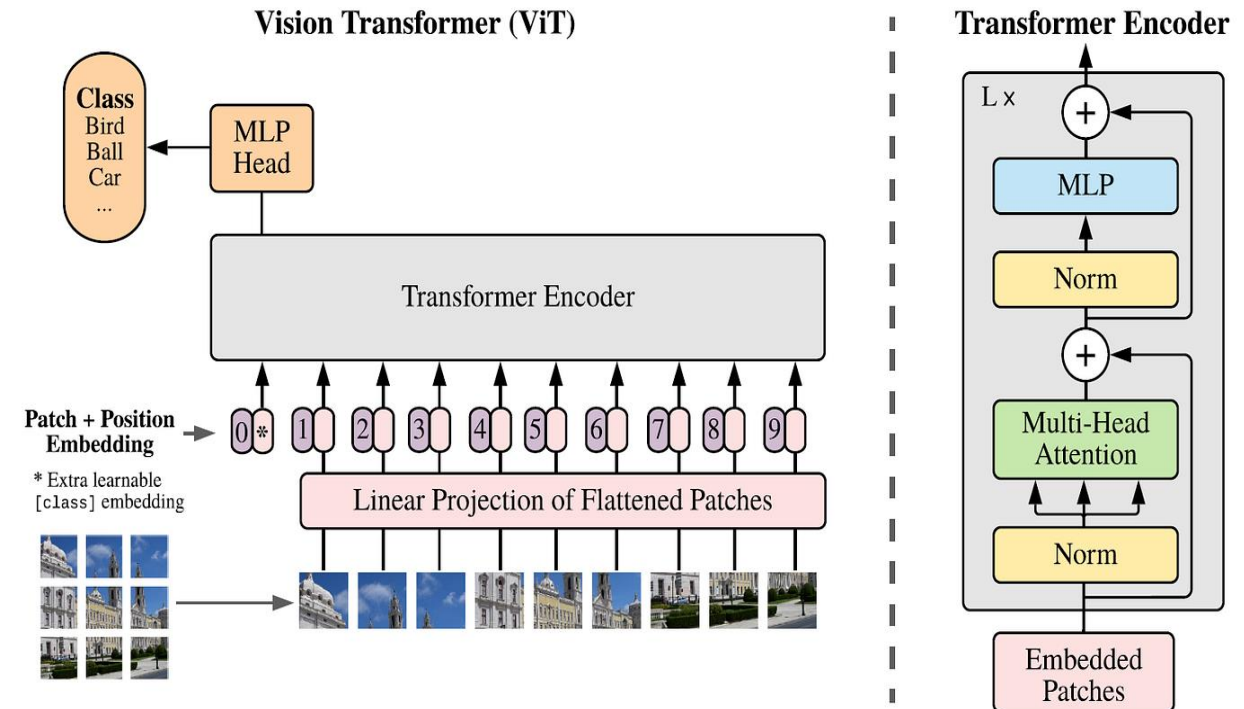


Image Encoder B: ViT

Vision Transformers

The state-of-the-art implementation swaps convolutions for **Vision Transformers (ViT)**. This architecture treats image patches exactly like words in a sentence. Images are sliced into non-overlapping patches (e.g., 16x16 pixels), projected into patch embeddings, and passed through transformer layers. This allows the model to use self-attention to map spatial dependencies across the entire canvas simultaneously, achieving better scaling efficiency.



Projection & Dimensional Realignment

A Problem of Dimensions

The raw output from the text encoder (d_t) and image encoder (d_v) rarely match. For example, the text model might emit a 512-D vector while the vision model emits a 768-D vector. These cannot be compared via dot product. CLIP solves this by introducing separate linear projection matrices (W_t and W_v).

Uniform Embedding Space

Both vectors are projected into a mutually agreed-upon dimension space (d_e). This shared space is where the "learning" happens. By forcing both modalities into this uniform bottleneck, the model finds the mathematical common ground between a pixel grouping and a linguistic concept.

Data Engineering: Web-Scale Curation

400M
Image-Text Pairs

The WIT Dataset

CLIP was trained on WIT (WebImageText), a massive curated dataset of 400 million pairs sourced from the public internet.

This scale is what makes the model robust.

Curation was focused on 500,000 distinct search queries to guarantee a balanced vocabulary. CLIP works not just because of its math, but because it has effectively memorized the visual-text layout of the internet.

The Contrastive Learning Shift



The Matching Puzzle

Traditional models try to predict exact pixels or tokens (generative). This is hard. CLIP solves a simpler matching puzzle.



Positive vs Negative

The objective is contrastive: Out of a batch of permutations, which caption belongs to which image?



Computational Efficiency

This shift is 4x to 10x more efficient at zero-shot classification than generative objectives, allowing for massive scaling.

Constructing the Similarity Matrix

Images \ Text	Caption 1	Caption 2	Caption N
Image 1	Match	Mismatch	Mismatch
Image 2	Mismatch	Match	Mismatch
Image N	Mismatch	Mismatch	Match

The $N \times N$ Matrix

During a training forward pass, CLIP processes a batch of N image-text pairs. It constructs an $N \times N$ matrix of similarity scores.

The network is optimized to maximize the cosine similarity of the N real pairs (the diagonal) while minimizing the scores of the $N^2 - N$ incorrect pairings (everywhere else).

Normalization & Matrix Dot Products

$$s_{i,j} = \left(\frac{g(I_i)}{\|g(I_i)\|} \right)^T \cdot \left(\frac{f(T_j)}{\|f(T_j)\|} \right)$$

Cosine Similarity

To ensure clean gradients, vectors are first **L2 normalized**. The similarity matrix is then computed via a highly parallelized matrix multiplication.

Because the vectors are normalized to unit length, the simple dot product directly yields the exact cosine similarity score between -1 and 1. This value tells us how semantically "aligned" the image is with the text.

The Symmetric Loss Philosophy

- ✓ **Dual Optimization:** CLIP optimizes a joint, symmetric cross-entropy loss function. It doesn't just train the vision model to understand text; it trains both to find a shared center.
- ✓ **Bi-Directional Alignment:** Pedagogically, "symmetric" means the model must solve two tasks: finding the correct text for an image AND finding the correct image for a text.
- ✓ **Preventing Collapse:** This dual path prevents "representation collapse," where the model might otherwise find a mathematical shortcut that ignores one of the modalities' nuances.

Why Symmetry Matters

Without symmetry, the embedding space might become warped towards one modality. By enforcing mutual agreement, the shared space stays balanced between structural visual properties and textual syntax.

InfoNCE Loss: Image-to-Text

The image-to-text classification loss for a single entry i treats row i of the similarity matrix as a classification task. The objective is to maximize the probability of the true label at diagonal index i .

$$L_{\text{img}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{i,i} \cdot e^{\text{T}})}{\sum_{j=1}^N \exp(s_{i,j} \cdot e^{\text{T}})}$$

Softmax Interpretation: The numerator isolates the true matching pair (the diagonal), while the denominator sums every incorrect combination in that row (the negative examples).

InfoNCE Loss: Text-to-Image

Column-wise Objective

The text-to-image loss operates identically but focuses on the columns of the similarity matrix:

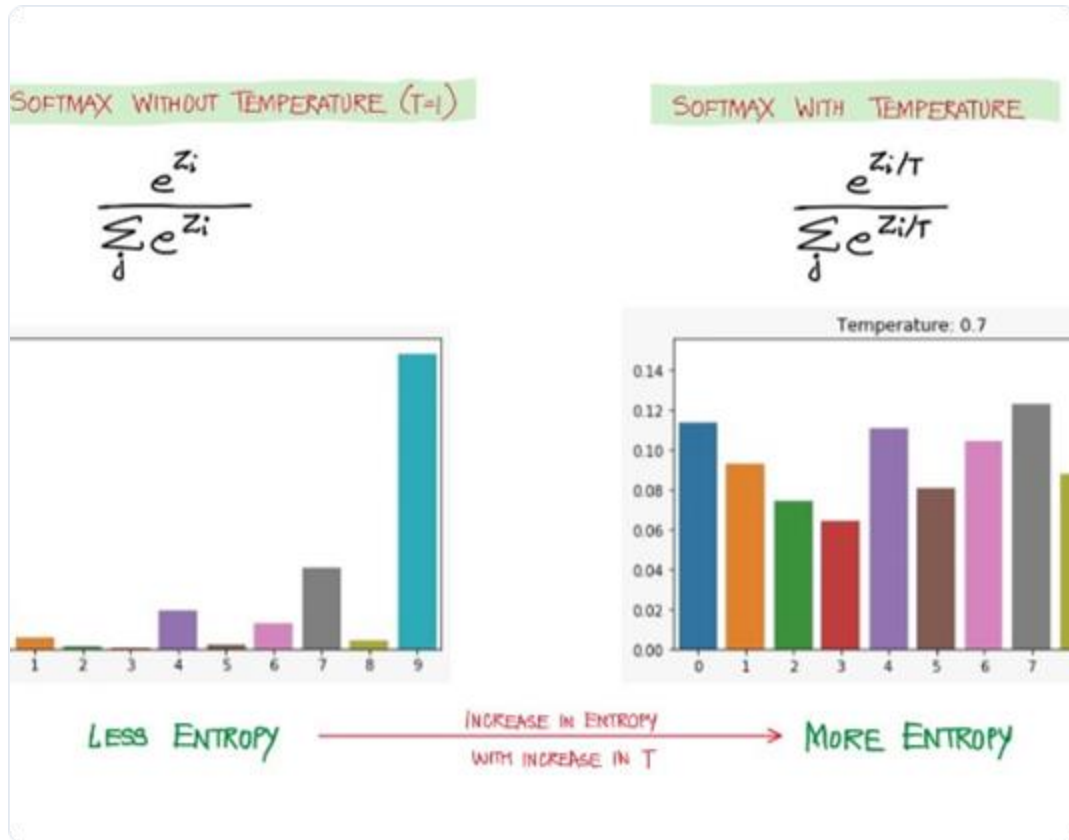
$$L_{\text{txt}} = -\frac{1}{N} \sum_{j=1}^N \log \frac{\exp(s_{j,j} \cdot e^{\top})}{\sum_{i=1}^N \exp(s_{i,j} \cdot e^{\top})}$$

The Final Circuit

The final optimized objective function is the scaled average of both pathways, completing the symmetric circuit:

$$L = \frac{1}{2} (L_{\text{img}} + L_{\text{txt}})$$

Learnable Temperature Parameter



- ✓ **Role of Tau:** The term τ represents a learnable log-scaled temperature parameter that scales the logits before the Softmax function is applied.
- ✓ **Controlling Sharpness:** This dynamically controls the "sharpness" of the probability distribution. If τ is too high, the distribution becomes too uniform (flat); if too low, it becomes hyper-sensitive (peaky).
- ✓ **Automated Optimization:** Making it a learnable parameter allows the network to optimize its own scaling sensitivity during backpropagation.

Optimization Mechanics at Scale



Extreme Scale

Trained across distributed nodes using a massive batch size of **32,768** instances.



1 Billion Pairings

The model evaluates over 1 billion contrastive combinations in every single forward pass.



Long-Cycle AdamW

Uses AdamW with a cosine learning rate decay over a multi-week training cycle.



Zero-Shot Classification Mechanics

CLIP's "Party Trick"

To classify images at inference time without any weight updates, we convert classification labels into text strings. This enables "Open-Vocabulary" recognition.

- ✓ **Step 1:** Pass the target image through the *Image Encoder* to get a visual vector.
- ✓ **Step 2:** Pass all candidate label strings (e.g., "dog", "cat", "plane") through the *Text Encoder*.
- ✓ **Step 3:** Compute the dot product between the image vector and each text vector.
- ✓ **Step 4:** The sentence with the highest mathematical alignment "wins" the classification.

The Art of Prompt Engineering

- ✓ **Polysemy Issues:** Passing single words like "cat" yields weak performance because they lack syntactic context.
- ✓ **Template Strategy:** Performance improves dramatically using templates like "*A photo of a [label]*" or "*A centered [label]*".
- ✓ **Context Alignment:** Just like BERT, CLIP needs context. These templates match the distribution of the internet captions it was trained on.

Why Context Matters

The text encoder was trained on full captions, not dictionary entries. Prompt engineering ensures the query "looks" like the data the model memorized, maximizing semantic alignment.

Visualizing Zero-Shot Inference



Candidate Prompt	Probability
"A photo of a dog"	0.01%
"A photo of a rocket"	1.59%
"A photo of a sports car"	98.40%

Final Calculation: Dot Product → Softmax → Top-1 Prediction

Linear Probe Evaluation Method

Diagnostic Method

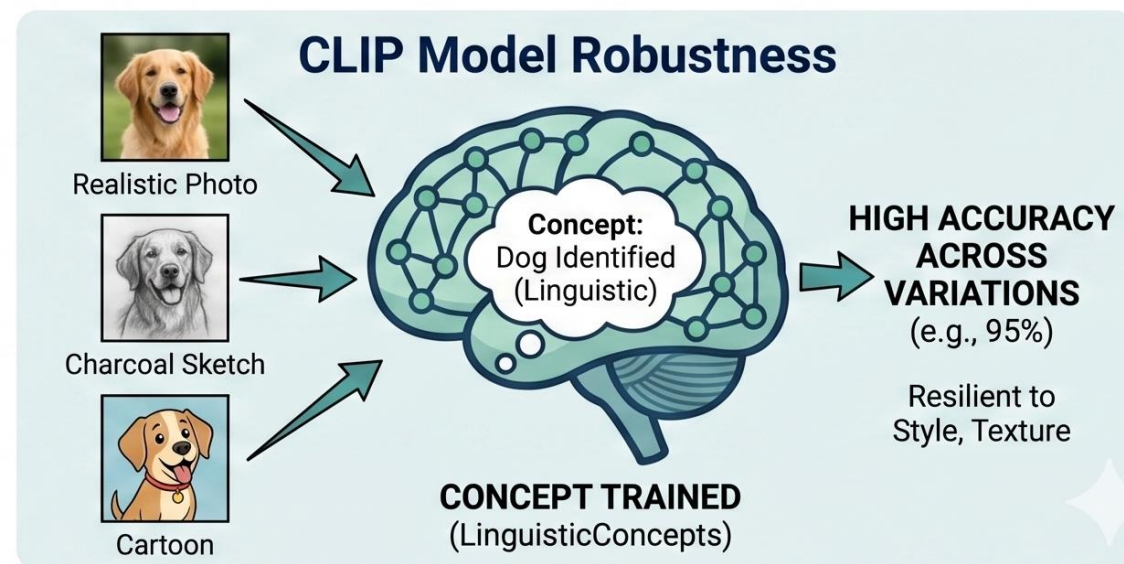
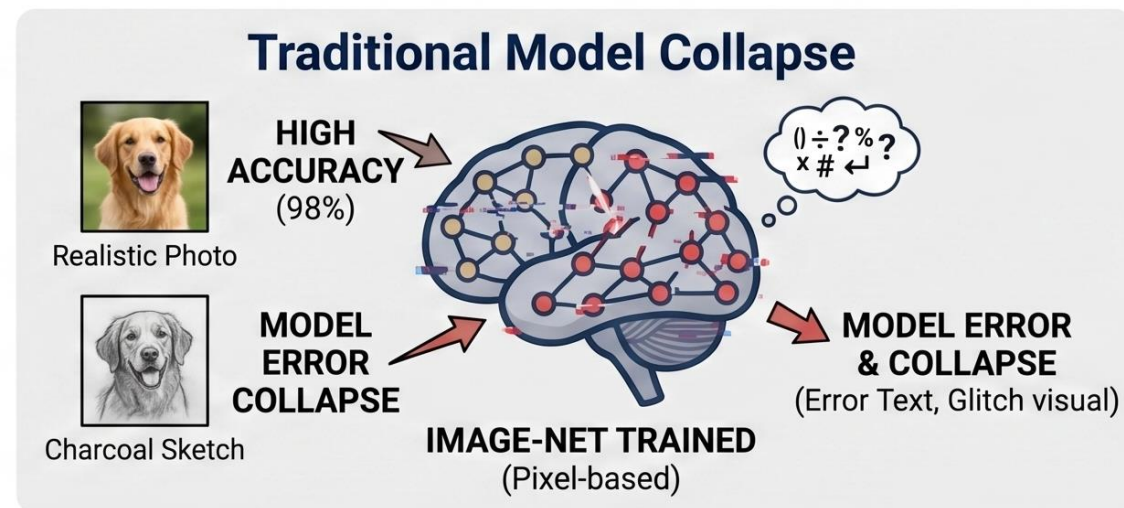
Alternatively, we "freeze" the Image Encoder and append a simple linear classification layer (logistic regression) on top of the visual embeddings.

Evidence of Separation

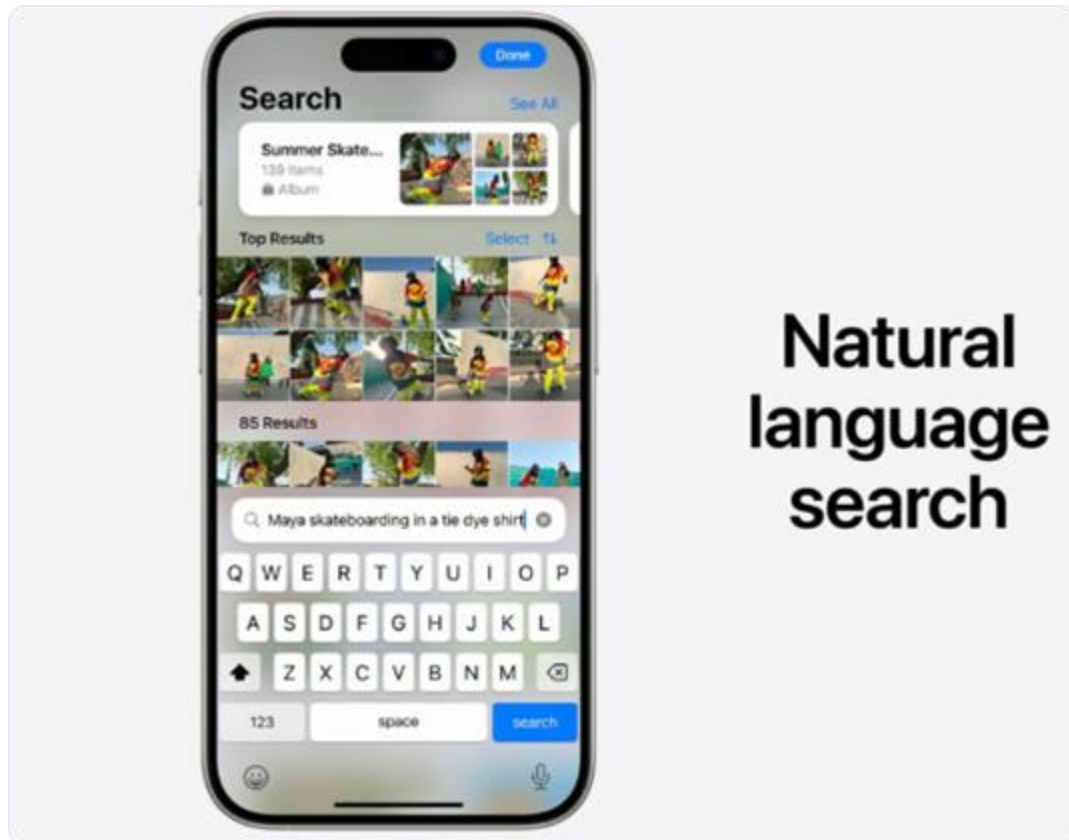
If this frozen model achieves high accuracy, it proves the embedding space *natively* separated visual concepts without needing dataset-specific fine-tuning.

OOD Robustness

- ✓ **Beyond Pixels:** Traditional models trained on ImageNet often collapse when tested on sketches or cartoons.
- ✓ **Abstract Abstractions:** CLIP preserves high accuracy across out-of-distribution (OOD) variations because it learned *linguistic concepts*.
- ✓ **Engineering Value:** It is remarkably resilient to changes in style, texture, and adversarial corruption.

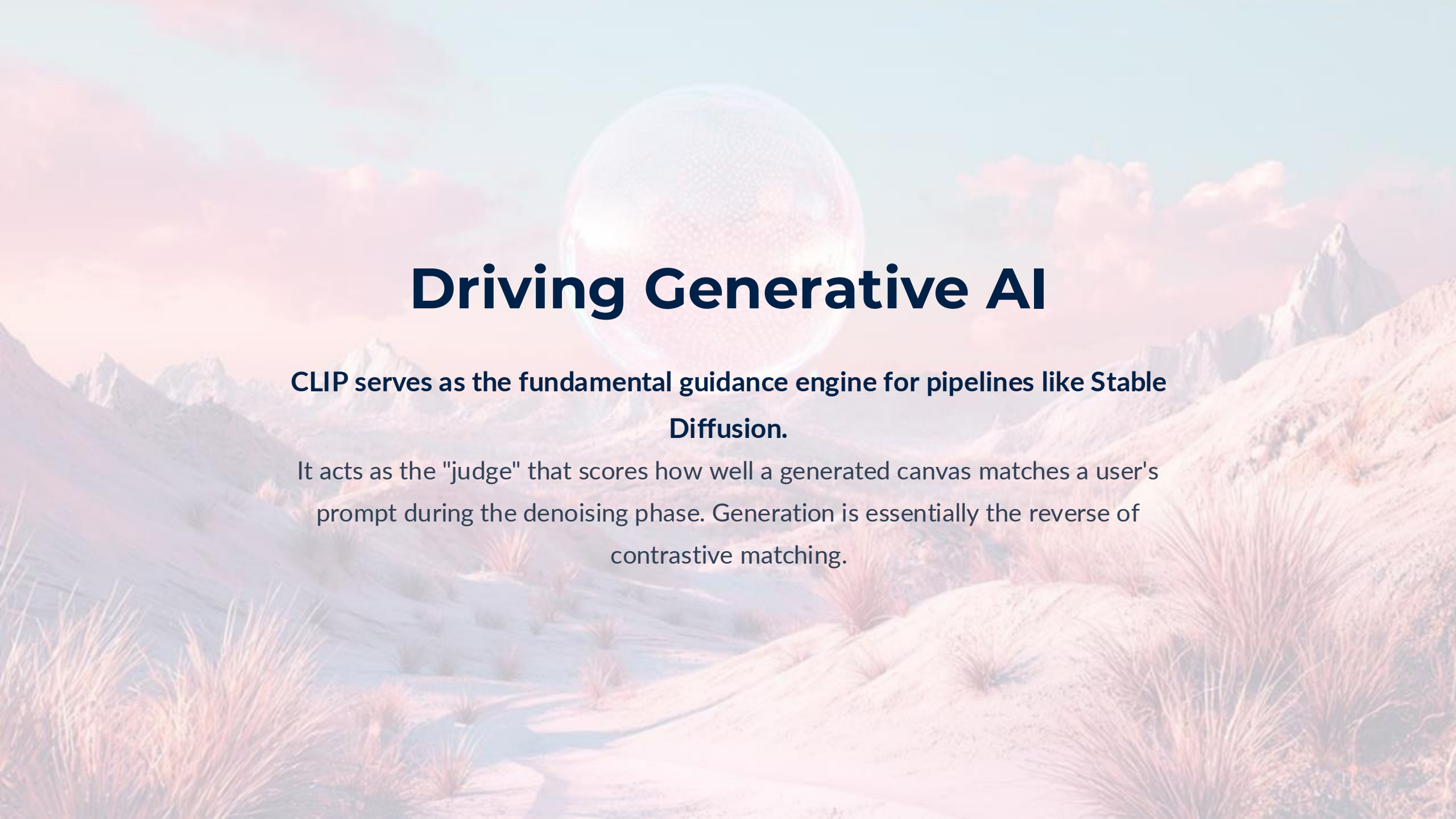


Semantic Image Retrieval



Natural Language Search

By indexing a database of CLIP image vectors, users can perform complex searches like "picnic by the lake last summer" without any manual tagging or metadata. This is the engine behind modern smartphone photo galleries and professional digital asset management systems.

A surreal landscape with a large, textured sphere in the sky and a path leading through dunes.

Driving Generative AI

CLIP serves as the fundamental guidance engine for pipelines like Stable Diffusion.

It acts as the "judge" that scores how well a generated canvas matches a user's prompt during the denoising phase. Generation is essentially the reverse of contrastive matching.

Limitations of CLIP

- ✓ **Spatial & Geometric Blindness:** Struggles to differentiate "red ball left of blue cube" from "blue ball left of red cube."
- ✓ **Counting Failure:** Consistently fails to identify exact counts of items (e.g., "cluster of seven pens").
- ✓ **Domain Specificity:** Lacks performance on highly technical fields, such as identifying specific cancer cells on a medical scan.
- ✓ **Bag-of-Concepts:** It often treats text as an unstructured bag of terms, overlooking fine-grained grammar and positional logic.

Final Takeaway: A powerful tool, but not a replacement for domain-specific specialized architectures.

Conclusion



Reviewing the theoretical journey and establishing the pragmatic choices in Applied NLP.

Summary and Conclusion

The Theoretical Journey

This lecture has mapped a comprehensive historical and theoretical journey of how machines process semantic meaning. The evolution proves that "**context**" is the driving force of intelligence—starting from surrounding words in a sentence, and expanding to the physical, visual world via mathematical mapping.

- ✓ **Foundations:** Word embeddings build upon the Distributional Hypothesis (DH) and Machine Learning (ML) to produce dense real-value vectors.
- ✓ **The Pioneer:** **word2vec** served as the first major milestone, proving the efficacy of local context prediction.
- ✓ **Dynamic Context:** Contextual embeddings like **ELMo** and **BERT** represent radical innovations in text-only modeling, solving polysemy dynamically.
- ✓ **Expanded Topologies:** New forms of embeddings cater to diverse data types, scaling up to entire documents (doc2vec) and graph levels (graph2vec).
- ✓ **The Multimodal Frontier:** **CLIP** and multimodal models represent the latest frontier, aligning visual pixels and textual semantics into a unified shared space.

Bag of Words: Trade-offs

While computationally inefficient for complex semantic tasks, BoW remains relevant because it works "out of the box" without needing millions of documents to train on.



Advantages

- Extremely easy to implement and conceptually understand.
- Highly interpretable (you can look exactly at which words triggered a classification).
- Does **not** require a massive, pre-existing training corpus or GPU infrastructure to operate.



Limitations

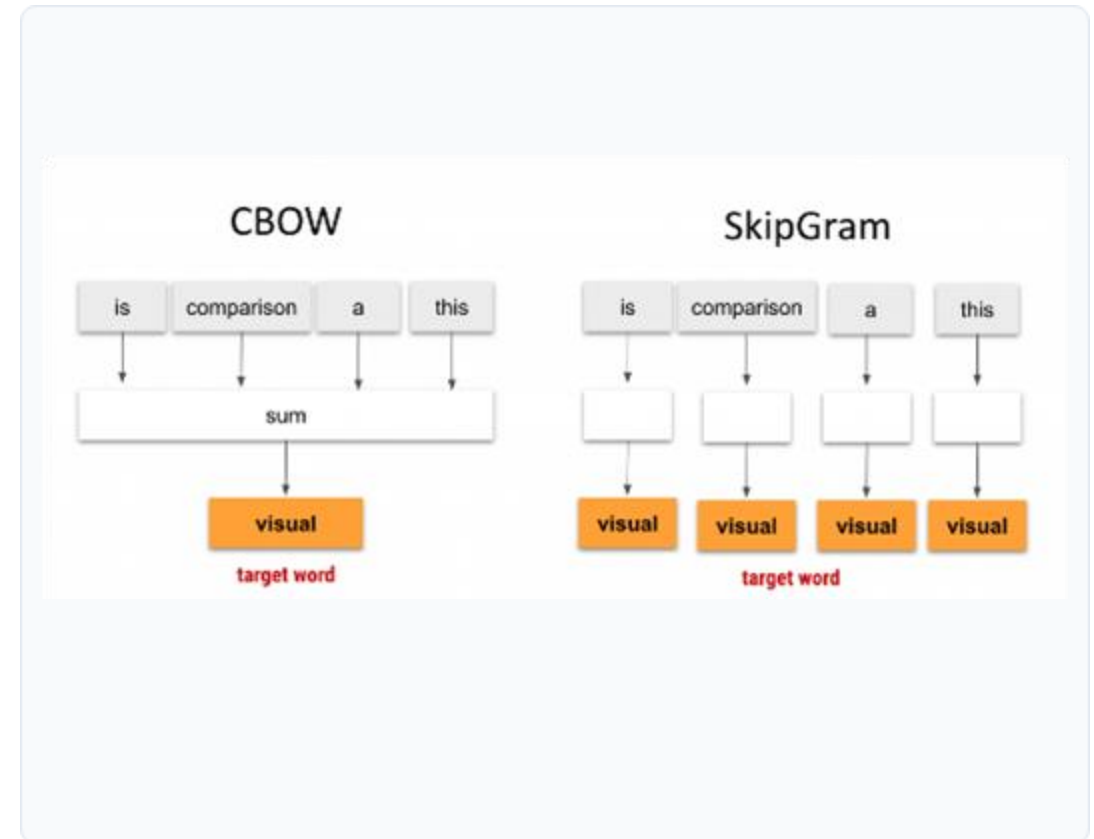
- Completely ignores semantic context ("bank" is always "bank").
- Destroys word order ("dog bites man" equals "man bites dog").
- Produces massive, sparse matrices that cause memory bottlenecks and suffer from the "curse of dimensionality".

The Pioneer: Word2Vec

Neural Network-Based Word Embeddings

Word2Vec represents the first major milestone of modern NLP. It is a predictive neural network technique designed to generate word embeddings by generating meaning strictly from immediate neighboring words (local context).

- ✓ **Key Characteristics:** Instead of sparse counts, it produces dense, continuous, low-dimensional vectors (e.g., 300 dimensions).
- ✓ **Semantic Mapping:** It captures syntactic and semantic meaning based on local context windows.
- ✓ **Common Uses:** Groundwork for word similarity tasks, clustering, semantic text classification, and acting as the foundational input layer for more complex sequential models (like RNNs).



Word2Vec: Trade-offs

Word2Vec dramatically improved computational efficiency and semantic tracking, but suffers from a critical flaw: it generates only one static vector per word.



Advantages

- Successfully captures complex semantic and syntactic relationships (e.g., King - Man + Woman = Queen).
- Produces compact, dense vectors, radically speeding up downstream machine learning training times.
- Relatively fast to train compared to modern deep transformer architectures.



Limitations

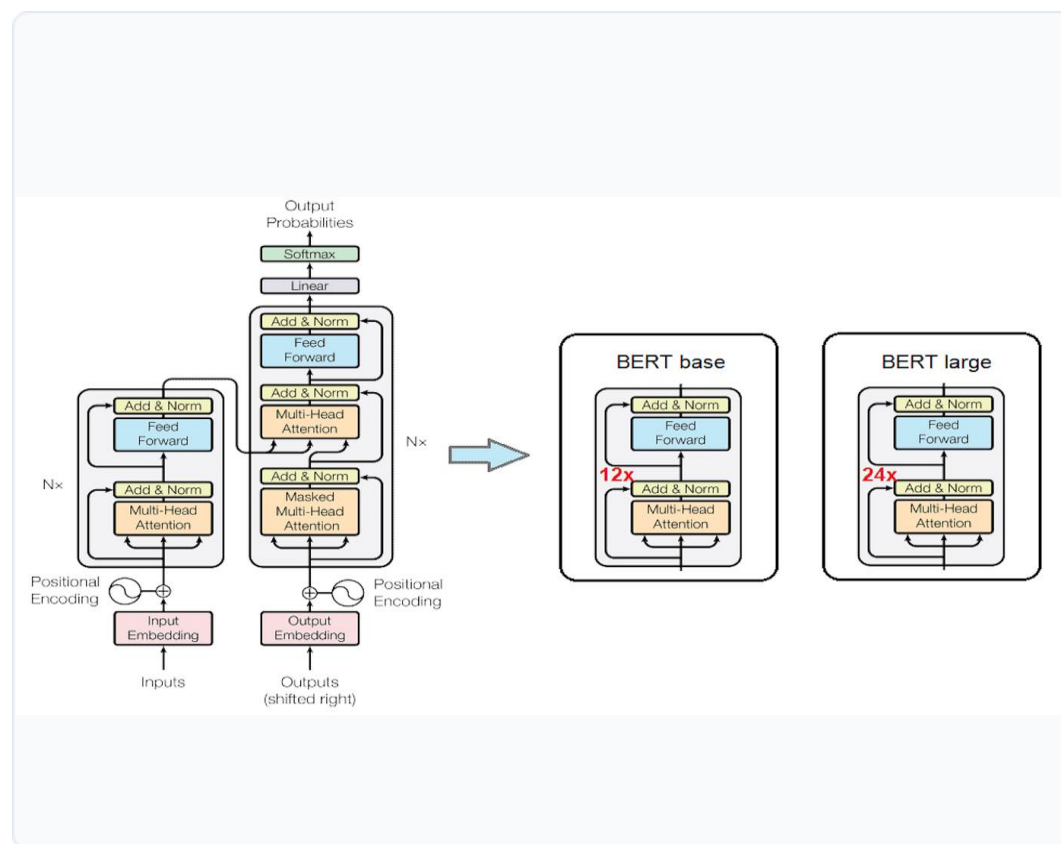
- **Polysemy Failure:** It is context-independent at inference. It cannot differentiate between distinct meanings of words (e.g., "financial bank" vs. "river bank" receive the exact same vector).
- Requires a very large textual corpus for initial training to develop accurate semantic coordinates.

The Contextual Era: BERT

Bidirectional Encoder Representations from Transformers

BERT introduces the state-of-the-art methodology. It is a transformer-based model designed specifically for generating dynamically contextualized word embeddings, fundamentally solving the context problem present in Word2Vec.

- ✓ **Key Characteristics:** Utilizes self-attention mechanisms to read text bidirectionally (left-to-right and right-to-left simultaneously), achieving deep contextual understanding.
- ✓ **Architecture:** Pre-trained on massive corpora and designed to be fine-tunable. It outputs dense, high-dimensional vectors.
- ✓ **Common Uses:** Advanced NLP tasks such as precise question answering, nuanced sentiment analysis, and named entity recognition.



BERT: Trade-offs

We must acknowledge the cost of high accuracy. While BERT is incredibly powerful, it requires immense amounts of computational power compared to legacy methods.



Advantages

- **Solves Polysemy:** Considers full sentence context dynamically, meaning the vector for "bank" changes based on the surrounding sentence.
- Delivers superior, state-of-the-art performance on complex linguistic and reasoning tasks.
- Can be fine-tuned on relatively small domain-specific datasets after its massive pre-training phase.



Limitations

- Extremely computationally intensive to train from scratch.
- Requires significant hardware resources (high VRAM GPUs) just for inference and deployment in production environments.
- "Black Box" nature makes it less interpretable than simpler models like Bag of Words.

Architectural Comparison Matrix

Feature	Bag of Words (BoW)	Word2Vec	BERT	CLIP
Contextual Information	No context.	Partial (local) context window.	Full sentence text context (bidirectional).	Full text context + Visual context .
Vector Type	Sparse, exceedingly high-dimensional.	Dense, low-dimensional static.	Dense, high-dimensional dynamic.	Dense, high-dimensional, Multimodal (shared space) .
Training and Usage	No pre-training needed. Count based.	Requires training on a large text corpus.	Massively Pre-trained, fine-tunable.	Pre-trained via contrastive learning, capable of Zero-Shot inference .
Primary Applications	Basic NLP classification tasks.	Similarity metrics, basic clustering.	Advanced NLP (QA, deep sentiment).	Cross-modal search, zero-shot visual classification, guiding generative AI.

Final Conclusion: The Pragmatic Choice



Selecting the Right Representation Technique

This comprehensive analysis highlights the differences and evolutionary applications from BoW, through Word2Vec and BERT, up to multimodal CLIP.

The final takeaway reinforces pragmatism: An analyst or engineer should not default to the newest or most complex tool (like BERT or CLIP) simply because it is state-of-the-art. You must choose the technique based on specific task requirements. If a simpler tool like BoW or Word2Vec solves your specific problem efficiently, it should be utilized to save on computational overhead, latency, and environmental resources.

References & Further Reading

Standard academic bibliography and documentation links for the architectures discussed in this session:

- ✓ **ELMo:** <https://allennlp.org/elmo>
- ✓ **BERT:** <https://github.com/google-research/bert>
- ✓ **Doc2Vec:** <https://radimrehurek.com/gensim/models/doc2vec.html>
- ✓ **Graph2Vec:** <https://github.com/benedekrozemberczki/graph2vec>
- ✓ **Tweet2Vec:** <https://github.com/bdhingra/tweet2vec>
- ✓ **Emoji2Vec:** <https://github.com/uclmr/emoji2vec>
- ✓ **CLIP (Foundational Paper):** Radford et al., 2021: "*Learning Transferable Visual Models From Natural Language Supervision*"
<https://github.com/OpenAI/CLIP>

Questions & Answers

Thank you for your attention.